

Rapport de projet DevSecOps — Notes App

RAPPORT DE PROJET

Intégration de contrôles de sécurité automatisés (DevSecOps) dans un pipeline CI/CD multi-niveaux GitHub Actions

Du poste du développeur au déploiement en production, sur l'application Flask « Notes App
»

Réalisé par : **Amen Saidani & Omar Abdelwahed**

Classe : 5NIDS2 | Module : DevSecOps | Date : octobre 2026

Dépôt : github.com/Omar-Abdelwahed/devsecops-project

Sommaire

1. Introduction et contexte
2. Objectifs
3. État des lieux (as-is)
4. Architecture cible (to-be)
5. Environnement et outils
6. Mise en œuvre étape par étape
7. Quality gates et Security Gate
8. Processus d'exemption
9. Reporting, alerting et historisation
10. Résultats
11. Difficultés rencontrées
12. Axes d'amélioration
13. Conclusion

Annexe A. Commandes de reproduction · Annexe B. Fichiers de configuration

1. Introduction et contexte

Une faille trouvée en production coûte bien plus cher qu'une faille arrêtée au moment du commit. Le DevSecOps applique donc le principe du *shift-left* : les contrôles de sécurité sont automatisés et déplacés au plus près du développeur, à chaque commit, chaque push et chaque pull request.

Ce projet construit une chaîne DevSecOps complète autour de « Notes App », une application web Python (Flask) de prise de notes écrite volontairement avec 10 failles : injection SQL, XSS stockée et réfléchie, injection de template, injection de commandes, secrets en dur, hachage MD5, mode debug, dépendances obsolètes et image Docker exécutée en root.

Contrairement à un simple audit, le projet va jusqu'au bout du cycle : les failles sont détectées, puis **corrigées dans le code**, et l'application corrigée est déployée automatiquement en staging puis en production. La chaîne est écrite en YAML pour GitHub Actions et découpée en six niveaux, sur le modèle Développement → CI/CD → Acceptation → Production. Elle combine des runners hébergés par GitHub et un runner auto-hébergé sur notre poste, où Docker Desktop fait tourner SonarQube, le staging et la production.

2. Objectifs

- Appliquer le *shift-left* : bloquer secrets et failles dès le poste du développeur (hooks pre-commit).
- Intégrer dans un pipeline réel la détection de secrets, le SAST, la SCA, le scan IaC, le scan d'image, le SBOM et le DAST.
- Mesurer l'état initial (as-is), puis imposer des quality gates qui bloquent toute faille HIGH ou CRITICAL corrigeable (to-be).
- Corriger réellement les failles de l'application, de ses dépendances et de son image Docker.
- Centraliser la décision dans un Security Gate unique, versionné et qui échoue par défaut.
- Déployer automatiquement en staging, attaquer l'application, puis déployer en production avec retour arrière.
- Documenter les faux positifs et les risques acceptés, produire des rapports lisibles et alerter l'équipe sur Discord.

3. État des lieux (as-is)

Le point de départ est l'application vulnérable et un premier pipeline en **mode audit** : les scanners tournent avec `continue-on-error: true` et `|| true`, donc le pipeline reste vert quoi qu'ils trouvent. Ce mode mesure la dette de sécurité sans bloquer l'équipe.

Faille	Emplacement	CWE	Contrôle qui la détecte
Injection SQL dans <code>/login</code>	<code>app.py</code> ligne 57	CWE-89	Bandit B608
Injection de commandes (<code>/ping</code>)	<code>app.py</code>	CWE-78	Bandit B605
XSS stockée et réfléchie, injection de template	<code>/notes</code> , <code>/search</code>	CWE-79, CWE-1336	Tests Pytest, ZAP (en-têtes)

Faible	Emplacement	CWE	Contrôle qui la détecte
Clé Flask et token en dur	app.py	CWE-798	Gitleaks, Bandit B105
Hachage MD5 sans sel	app.py	CWE-327	Bandit B324, SonarQube S4790
Mode debug activé	app.run(debug=True)	CWE-489	Bandit B201, SonarQube S4507
Flask et Werkzeug 2.0.1	requirements.txt	CWE-1104	Trivy fs : 3 CVE HIGH
Image python:3.8 en root, COPY . .	Dockerfile	CWE-250	Checkov, SonarQube S6470/S6471
Protection CSRF absente	routes POST	CWE-352	SonarQube S4502

Premiers résultats mesurés sur le code d'origine : Bandit 7 alertes (3 HIGH, 2 MEDIUM, 2 LOW) ; Trivy 3 CVE HIGH ; SonarQube note de sécurité E avec 6 vulnérabilités ; Checkov 3 échecs.

Dès que le mode audit a été retiré, le pipeline est passé au rouge : la figure 1 montre le contrôle SCA (Trivy) bloquant la livraison à cause des dépendances obsolètes, ce qui empêche aussi la construction de l'image et le DAST.

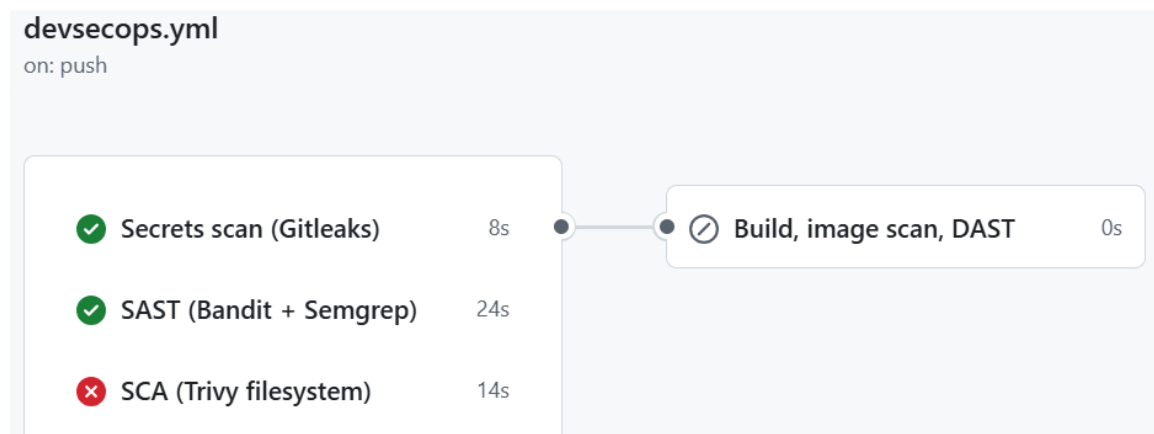


Figure 1 : quality gates activés, le job SCA (Trivy) échoue sur les CVE de Flask et Werkzeug ; le build et le DAST ne sont pas lancés.

4. Architecture cible (to-be)

L'architecture cible suit quatre zones : **Développement** (contrôles sur le poste), **CI/CD** (analyses automatiques), **Acceptation** (Security Gate, staging et DAST) et **Production** (déploiement automatique). La figure 2 résume l'ensemble : l'environnement du développeur, le dépôt, les étapes du pipeline, les rapports, les alertes et les composants d'infrastructure.

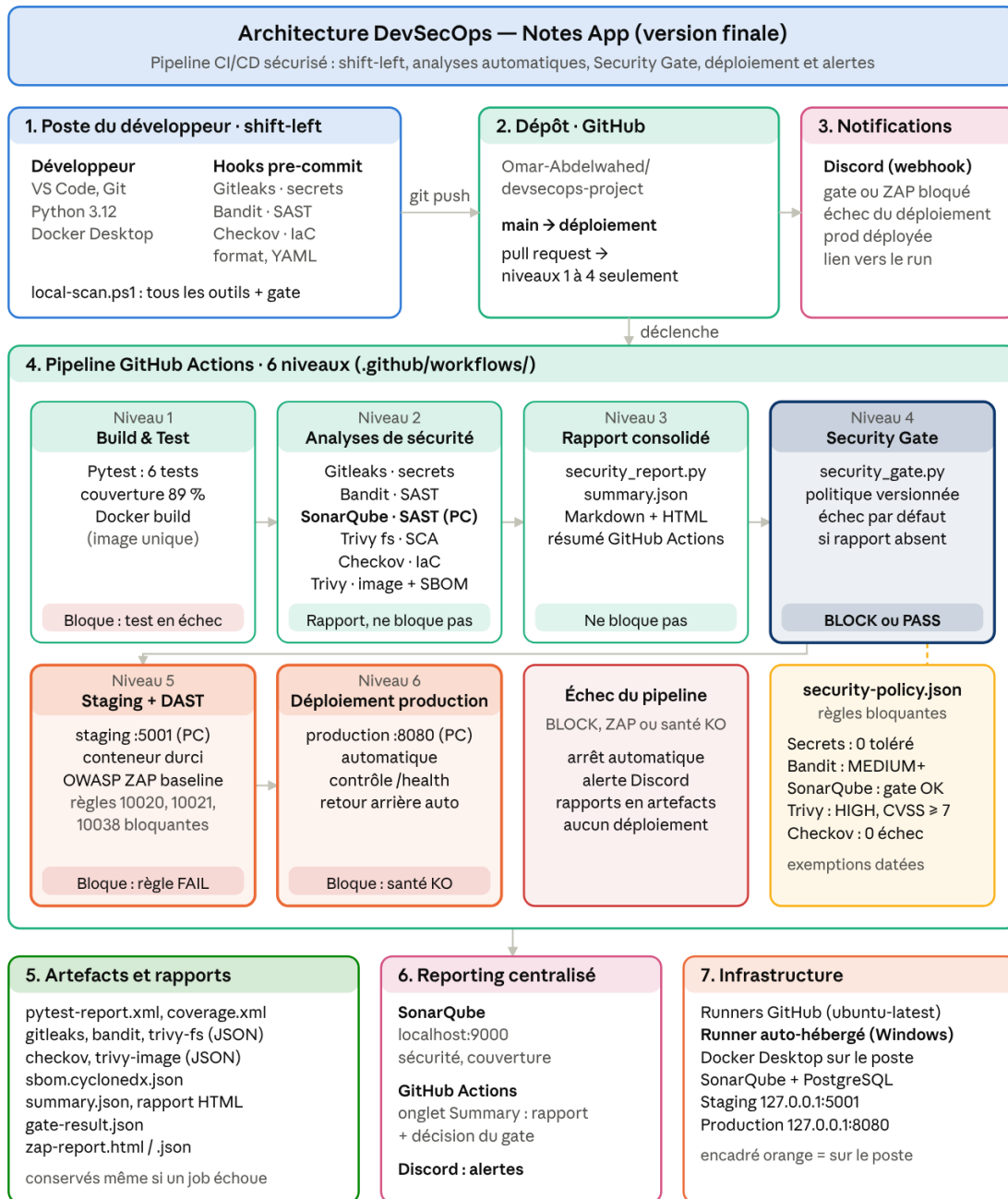


Figure 2 : architecture DevSecOps complète : poste du développeur, dépôt GitHub, notifications, pipeline en 6 niveaux, artefacts, reporting et infrastructure. Les blocs encadrés en orange s'exécutent sur notre poste via le runner auto-hébergé.

Chaque niveau du pipeline est un workflow réutilisable, enchaîné par l'orchestrateur `.github/workflows/devsecops.yml` ; un niveau ne démarre que si le précédent a réussi.

Niveau	Workflow	Outils	Exécution	Bloquant
1 Build & Test	level1-build-test.yml	Pytest + couverture, Docker build	Runner GitHub	Oui (test en échec)

Niveau	Workflow	Outils	Exécution	Bloquant
2 Security Scans	level2-security-scans.yml	Gitleaks, Bandit, SonarQube, Trivy fs, Checkov, Trivy image + SBOM	Runner GitHub ; SonarQube sur le poste	Non : mode rapport
3 Report Generation	level3-reports.yml	scripts/security_report.py	Runner GitHub	Non
4 Security Gate	level4-security-gate.yml	scripts/security_gate.py + security-policy.json	Runner GitHub	Oui : BLOCK ou PASS
5 Staging + DAST	level5-staging-dast.yml	Docker, OWASP ZAP	Poste (Docker Desktop)	Oui (règle ZAP FAIL)
6 Deploy to Production	level6-deploy-production.yml	Docker	Poste (Docker Desktop)	Oui (contrôle de santé, retour arrière)
Alertes	notify-discord.yml	Webhook Discord	Runner GitHub	Non

Deux règles structurent le pipeline. **Les scanners rapportent, le gate décide** : au niveau 2 aucun outil ne fait échouer le pipeline, la décision est prise une seule fois au niveau 4 selon une politique versionnée. **L'image est construite une seule fois** : celle qui est analysée, testée en staging et déployée en production est la même, jamais reconstruite. Les pull requests exécutent les niveaux 1 à 4 ; seul un push sur `main` déploie.

5. Environnement et outils

Outil	Rôle dans le projet
Git, GitHub	Dépôt, historique, pull requests
GitHub Actions	Pipeline : runners hébergés ubuntu-latest et un runner auto-hébergé Windows (label docker-desktop)
Docker, Docker Desktop	Build de l'image ; staging et production ; chaque scanner tourne aussi en conteneur, ce qui rend les analyses reproductibles en local
Python 3.12, Flask, Gunicorn, SQLite	Application cible « Notes App »
Pytest, pytest-cov	6 tests dont 3 tests de sécurité, couverture de code
pre-commit	Contrôles exécutés à chaque commit sur le poste
Gitleaks	Détection de secrets
Bandit	SAST du code Python

Outil	Rôle dans le projet
SonarQube Community (+ PostgreSQL)	SAST, qualité et couverture ; serveur dans Docker Desktop
Trivy	SCA des dépendances, scan de l'image, SBOM CycloneDX
Checkov	Scan IaC du Dockerfile et des workflows GitHub Actions
OWASP ZAP	DAST baseline contre le staging
Discord (webhook)	Alertes du pipeline
VS Code, PowerShell	Poste de développement sous Windows 11

Choix des outils. Nous avons appliqué une règle simple : ne pas ajouter d'outil quand un outil déjà présent couvre le besoin. Trivy génère donc le SBOM à la place de Syft, et Checkov remplace Hadolint car il couvre le Dockerfile **et** les workflows. Semgrep, utilisé au début, a été remplacé par SonarQube, qui a trouvé une vraie faille (CSRF) manquée par les autres outils.

6. Mise en œuvre étape par étape

Étape 1 : dépôt et structure du projet

Le dépôt sépare l'application, le pipeline et la configuration des outils. Le pipeline se déclenche sur chaque push vers `main`, chaque pull request et à la demande ; tous les workflows n'ont que la permission `contents: read`.

devsecops-project/	
app/app.py	application Flask
tests/	tests Pytest (dont SQLi, XSS, en-têtes)
Dockerfile, .dockerignore	image durcie
.github/workflows/	orchestrateur + un fichier par niveau
security-policy.json	seuils du Security Gate
scripts/	rapport, gate, déploiement, scan local
bandit.yaml, .checkov.yaml	configuration et exemptions des scanners
.trivyignore, .zap/rules.tsv	
sonarqube/docker-compose.yml	serveur SonarQube + PostgreSQL
.pre-commit-config.yaml	contrôles avant commit
exemption-process.md	registre des exemptions

Étape 2 : contrôles sur le poste (pre-commit)

Quatre contrôles s'exécutent automatiquement à chaque `git commit` : Gitleaks (secrets), Bandit (failles Python MEDIUM et plus), Checkov dans Docker (Dockerfile et workflows) et des vérifications de format. Si l'un échoue, le commit est refusé avant d'entrer dans l'historique Git. Ils utilisent les mêmes fichiers de configuration que le pipeline. Le script `scripts/local-scan.ps1` lance en plus tous les scanners, le rapport et le Security Gate sur le poste, avec Docker seulement.

```

PS C:\Users\malak\Desktop\devsecops> git commit -m "Changed the pipeline to multilevel (Checkov, Central security gate, staging, production)"
[INFO] Locking pre-commit directory
check yaml.....Passed
fix end of files.....Passed
trim trailing whitespace.....Passed
bandit (SAST).....(no files to check)Skipped
checkov (IaC).....Passed
Detect hardcoded secrets.....Passed
[feat/remediation-sonarqube 9566987] Changed the pipeline to multilevel (Checkov, Central security gate, staging, production)
27 files changed, 1159 insertions(+), 356 deletions(-)
create mode 100644 .checkov.yaml

```

Figure 3 : commit de la nouvelle architecture : les hooks pre-commit (YAML, fins de fichier, Bandit, Checkov, Gitleaks) passent avant l'enregistrement du commit.

Étape 3 : build et tests (niveau 1)

Pytest exécute 6 tests, dont 3 tests de sécurité : l'injection admin ' -- ne contourne pas la connexion, une charge XSS dans /search est échappée, et les trois en-têtes de sécurité sont présents. La couverture atteint 89 %. L'image Docker est ensuite construite une seule fois et transmise aux niveaux suivants.

Étape 4 : détection de secrets avec Gitleaks

Gitleaks analyse tout le dépôt à la recherche de clés, tokens et mots de passe. Le code d'origine contenait une clé Flask, un token GitHub et une clé AWS en dur. Ils ont été retirés : la clé Flask est désormais lue dans une variable d'environnement et l'application refuse de démarrer sans elle. Résultat final : 0 secret.

Étape 5 : SAST avec Bandit

Bandit a trouvé 7 alertes sur le code d'origine, dont l'injection SQL B608 (CWE-89), l'injection de commandes B605, MD5 (B324) et le mode debug (B201). Correctifs : requêtes paramétrées, hachage salé de Werkzeug, suppression de la route /ping et du mode debug. Il reste une seule alerte, B104 (écoute sur 0.0.0.0), traitée comme faux positif (section 8).

Leçon : au premier passage en mode bloquant, Bandit était vert alors que le code contenait une injection SQL. La commande visait app.py à la racine, un fichier inexistant : Bandit n'analysait rien. Depuis, chaque contrôle est vérifié sur du code vulnérable connu.

Étape 6 : SAST et qualité avec SonarQube

SonarQube Community tourne dans Docker Desktop (avec PostgreSQL), sur le poste. Les runners de GitHub ne pouvant pas joindre localhost:9000, le poste est enregistré comme **runner auto-hébergé** : seul ce job s'y exécute. Un Quality Gate personnalisé « DevSecOps » échoue si la note de sécurité ou de fiabilité est pire que C (au moins une alerte HIGH ou BLOCKER) ou si la couverture est inférieure à 70 %.

Sur la version d'origine (figure 4), le gate échoue : note de sécurité E, 6 vulnérabilités, couverture 0 %.

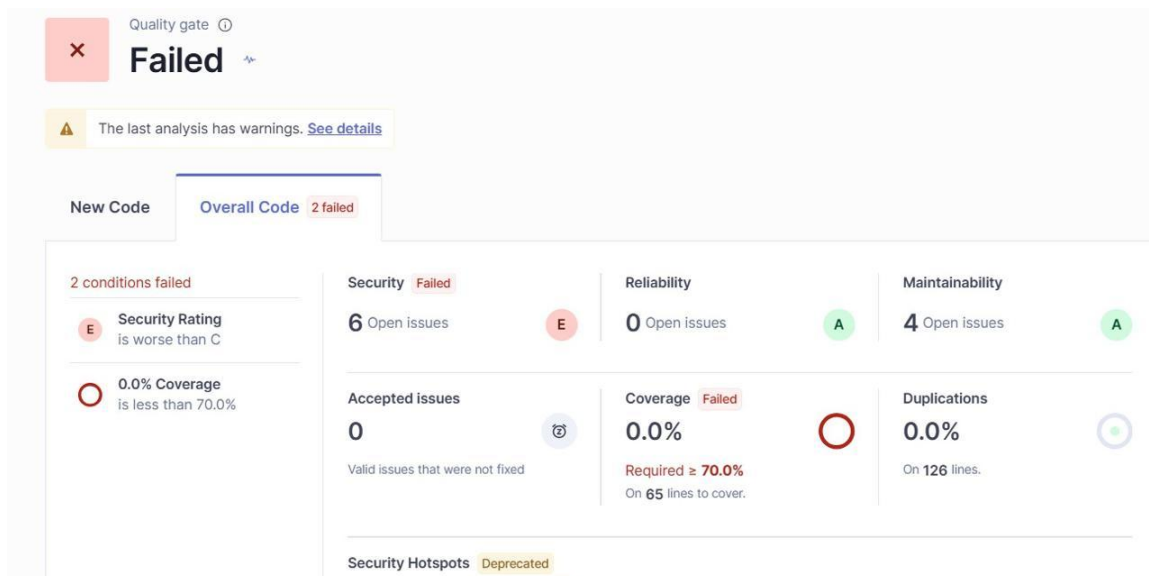


Figure 4 : SonarQube, version d'origine : Quality Gate échoué (note de sécurité E, 6 vulnérabilités, couverture 0 %).

Après remédiation, SonarQube a encore relevé deux points : l'écoute sur 0.0.0.0 (S8392, faux positif) et l'absence de protection CSRF (S4502), une vraie faille que Bandit n'avait pas vue, acceptée comme risque documenté. Le gate passe ensuite (figures 5 et 6).

DevSecOps Flask App / Overview

Overview

88 Lines of Code • Version not provided • Last analysis 10 minutes ago

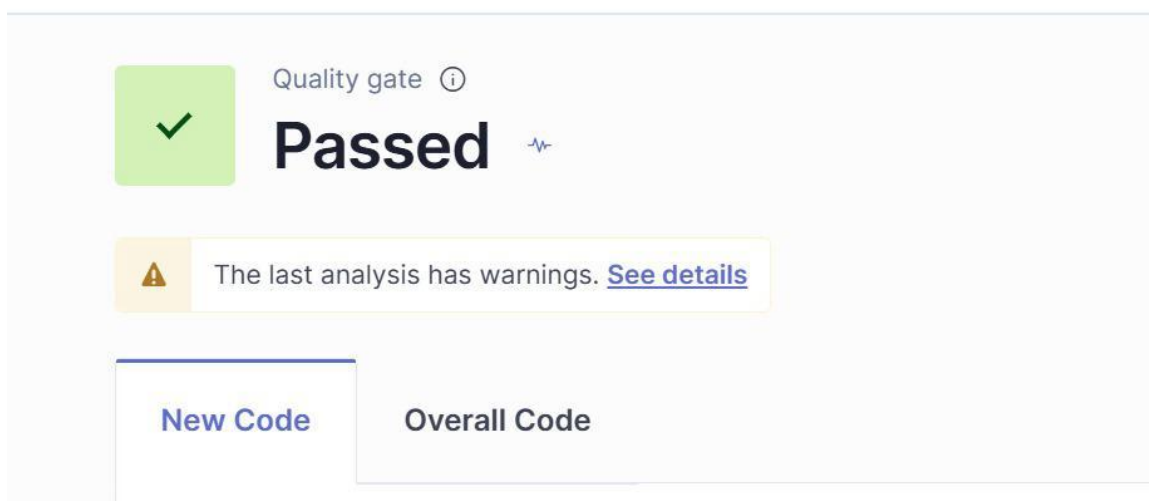


Figure 5 : Quality Gate « DevSecOps » réussi après remédiation.

Overview

88 Lines of Code · Version not provided · Last analysis 4 minutes ago

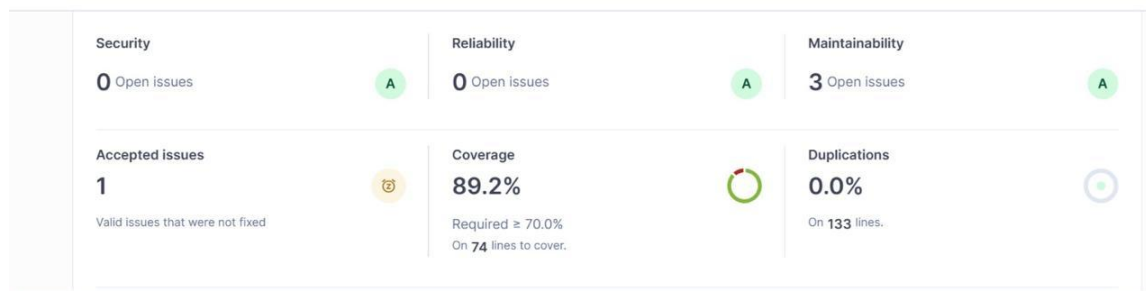
[Set as homepage](#)

Figure 6 : après remédiation : sécurité A, fiabilité A, couverture 89,2 %, 1 risque accepté (CSRF, EXC-002).

Limite constatée : SonarQube Community ne détecte ni l'injection SQL ni l'injection de commandes, car il faudrait l'analyse de teinte, réservée aux éditions payantes. Bandit et SonarQube se complètent donc.

Étape 7 : SCA avec Trivy

Trivy compare les versions de `requirements.txt` aux bases de CVE. Sur la version d'origine, trois CVE HIGH étaient corrigibles :

CVE	Paquet	CVSS 3.1	Description	Atteignable dans l'application ?
CVE-2023-25577	werkzeug 2.0.1 → 2.2.3	7,5	Déni de service par formulaire multipart géant	Oui : /login et /register lisent les formulaires sans authentification
CVE-2024-34069	werkzeug 2.0.1 → 3.0.3	7,5	Exécution de code via le débogueur	Oui dans la version d'origine (debug=True), non après correction
CVE-2023-30861	flask 2.0.1 → 2.3.2	7,5	Fuite du cookie de session via un cache	Non : ni proxy de cache ni session permanente

Correctif : passage à Flask 3.1.3 et Werkzeug 3.1.9, avec toutes les dépendances épinglées (==) et pytest déplacé dans `requirements-dev.txt`. Résultat : 0 CVE bloquante.

Étape 8 : scan IaC avec Checkov

Checkov audite le `Dockerfile` et les workflows GitHub Actions. Sur la version d'origine, il signale 3 échecs : conteneur exécuté en root (CKV_DOCKER_3), absence de HEALTHCHECK (CKV_DOCKER_2) et workflow sans restriction de permissions (CKV2_GHA_1). Après correction : 48 contrôles Dockerfile et 320 contrôles de workflows réussis, 0 échec. Checkov remplace Hadolint, qui avait

signalé en cours de projet une commande HEALTHCHECK au mauvais format et un utilisateur non numérique, tous deux corrigés.

Étape 9 : scan de l'image et SBOM avec Trivy

L'image d'origine (python:3.8 complet, exécutée en root, COPY . .) a été remplacée par une image durcie : python:3.12-slim, utilisateur non-root UID 10001, copie de app/ seulement, .dockerignore, HEALTHCHECK et Unicorn. Trivy a ensuite signalé 4 CVE HIGH (urllib3, msgpack, setuptools) qui ne venaient pas de nos dépendances mais des bibliothèques embarquées dans pip lui-même ; pip étant inutile à l'exécution, il est désinstallé à la fin du build. Résultat : 0 CVE bloquante. Trivy produit aussi le SBOM au format CycloneDX, inventaire de tous les composants de l'image.

Étape 10 : rapport consolidé (niveau 3)

Le script scripts/security_report.py lit les six rapports bruts (formats différents) et les convertit dans un format commun : outil, sévérité, règle, emplacement, score CVSS et version corrigée. Il produit summary.json pour le Security Gate, ainsi qu'un rapport Markdown et HTML affiché dans le résumé de l'exécution.

Étape 11 : Security Gate (niveau 4)

Le script scripts/security_gate.py applique la politique security-policy.json (section 7) et répond **BLOCK** ou **PASS**. Il échoue par défaut : un rapport absent, par exemple parce qu'un scanner a planté, bloque le pipeline. Pour prouver que le gate fonctionne, nous l'avons exécuté sur la version vulnérable : il bloque avec 4 alertes Bandit, 8 CVE et 3 échecs Checkov, alors qu'il laisse passer la version corrigée.

Étape 12 : staging et DAST avec OWASP ZAP (niveau 5)

Sur le poste, le runner auto-hébergé déploie l'image analysée dans le conteneur devsecops-staging (http://127.0.0.1:5001), durci : système de fichiers en lecture seule, aucune capacité Linux, no-new-privileges, limites mémoire et processus, port accessible depuis le poste seulement. OWASP ZAP rejoint le réseau Docker du staging et analyse l'application en fonctionnement ; les règles 10020 (anti-clickjacking), 10021 (X-Content-Type-Options) et 10038 (CSP) sont bloquantes. Résultat : 0 FAIL, 63 règles réussies, 4 avertissements non bloquants.

Notes App

Hello guest

[Notes](#)

Figure 7 : « Notes App » déployée par le pipeline dans l'environnement de staging (<http://127.0.0.1:5001>).

Étape 13 : déploiement en production (niveau 6)

Si le staging et ZAP réussissent, la même image est déployée automatiquement dans devsecops-production (<http://127.0.0.1:8080>), avec le même durcissement. Le script `scripts/deploy.ps1` met l'ancienne version de côté ; si la nouvelle ne démarre pas ou échoue au contrôle de santé `/health`, elle est supprimée et l'ancienne redémarre. Ce retour arrière a été testé en déployant volontairement une image défectueuse. La clé secrète de production est générée une seule fois sur le poste, lisible par ce seul compte Windows, et n'est jamais stockée dans Git ni dans GitHub.

Une approbation manuelle était prévue dans l'architecture de référence ; nous l'avons remplacée par un déploiement automatique, puisque la production ne reçoit une version qu'après le Security Gate et le DAST réussis. Une validation humaine peut être réactivée en ajoutant un relecteur obligatoire à un environnement GitHub.

Étape 14 : alertes Discord

Le dernier job envoie un message Discord avec la raison et le lien vers l'exécution : blocage au niveau 1, blocage du Security Gate, blocage par ZAP, échec du déploiement ou déploiement réussi. L'URL du webhook est stockée dans le secret GitHub `DISCORD_WEBHOOK_URL` ; s'il est absent, la notification est ignorée sans faire échouer le pipeline.



Figure 8 : pipeline multi-niveaux en cours d'exécution : niveaux 1 à 4 réussis (dont SonarQube sur le poste), niveau 5 en train de déployer le staging.

7. Quality gates et Security Gate

Le projet a connu deux modèles de quality gates. Dans le premier, chaque scanner bloquait lui-même le pipeline (`--exit-code 1`, `--error`). Dans l'architecture finale, les scanners produisent leurs rapports et un **Security Gate central** décide, à partir d'une politique unique versionnée dans `security-policy.json` : changer un seuil revient à modifier un fichier, relu dans une pull request.

Contrôle	Seuil de blocage	Justification
Gitleaks	Tout secret détecté	Un secret exposé est exploitable immédiatement et doit être révoqué
Bandit	Alerte MEDIUM ou HIGH non exemptée	L'injection SQL (B608) est classée MEDIUM : un seuil HIGH

Contrôle	Seuil de blocage	Justification
		l'aurait laissée passer
SonarQube	Quality Gate « DevSecOps » en échec	Note de sécurité ou fiabilité pire que C, ou couverture < 70 %
Trivy fs et Trivy image	CVE corrigeable HIGH/CRITICAL, ou score CVSS $\geq 7,0$	Un correctif existe : le blocage est actionnable
Checkov	Tout contrôle IaC échoué non exempté	Une mauvaise configuration se corrige dans le fichier
Tous les outils	Rapport absent	Échec par défaut : un scanner qui plante ne laisse rien passer
OWASP ZAP (niveau 5)	Règles 10020, 10021, 10038	En-têtes de sécurité obligatoires ; les avertissements ne bloquent pas (-I)
Déploiement (niveau 6)	Contrôle de santé en échec	Retour arrière automatique vers la version précédente

Choix des seuils. Nous bloquons dès HIGH, et non seulement CRITICAL : notre application est corrigée, donc un pipeline vert reste atteignable. Le seuil CVSS $\geq 7,0$ complète la sévérité donnée par l'éditeur : sur la version d'origine, il a bloqué 5 CVE classées seulement MEDIUM mais notées 7,0 ou plus (Jinja2, Werkzeug). L'option `--ignore-unfixed` reste appliquée : une CVE sans correctif est signalée mais ne bloque pas, faute de version vers laquelle mettre à jour.

Test du gate	Gitleaks	Bandit	Trivy fs	Checkov	Décision
Version corrigée	0	0	0	0	PASS
Version vulnérable d'origine	0	4 bloquantes	8 bloquantes	3 bloquantes	BLOCK

8. Processus d'exemption (faux positifs et risques acceptés)

Avec des seuils stricts, un faux positif bloque toute l'équipe. Sans processus clair, la tentation serait de désactiver le scanner ou de remettre `continue-on-error`, ce qui annulerait tout le travail. Une exemption est donc toujours ciblée, justifiée, relue et datée :

1. Analyser l'alerte et démontrer pourquoi elle n'est pas exploitable (faux positif) ou quel contrôle compense le risque (risque accepté).
2. Choisir le mécanisme le plus ciblé possible.
3. Inscrire l'exemption dans le registre `exemption-process.md` (identifiant EXC-xxx) et la référencer dans le fichier de configuration.
4. Faire relire le changement dans une pull request.
5. Réexaminer l'exemption à sa date de révision, puis la supprimer ou la renouveler.

ID	Alerte	Décision	Justification	Mécanisme	Révision
----	--------	----------	---------------	-----------	----------

ID	Alerte	Décision	Justification	Mécanisme	Révision
EXC-001	Bandit B104 (MEDIUM) et SonarQube S8392 (BLOCKER) : écoute sur 0.0.0.0	Faux positif	Dans un conteneur, écouter sur 127.0.0.1 rendrait l'application injoignable ; l'exposition réelle est contrôlée par Docker	skips: [B104] dans bandit.yaml ; statut « Faux positif » dans SonarQube	2027-04-08
EXC-002	SonarQube S4502 (HIGH) : protection CSRF absente	Risque accepté	Le cookie de session SameSite=Lax bloque les requêtes POST venant d'un autre site ; risque résiduel : le <i>login CSRF</i>	Statut « Accepté » dans SonarQube ; correctif prévu avec Flask-WTF	2027-01-08

Outil	Mécanisme d'exemption
Bandit	# nosec BXXX sur une ligne, ou skips dans bandit.yaml
SonarQube	Statut « Faux positif » ou « Accepté » avec commentaire obligatoire, ou # NOSONAR
Trivy	Fichier .trivyignore, une CVE par ligne avec date d'expiration (vide aujourd'hui)
Checkov	skip-check dans .checkov.yaml ou # checkov:skip= dans le fichier
Gitleaks	Fichier .gitleaksignore (empreinte exacte)
OWASP ZAP	Règle passée en IGNORE dans .zap/rules.tsv

Ces fichiers sont lus par les scanners eux-mêmes : une exemption s'applique pareil sur le poste et dans le pipeline. Le Security Gate ne contient aucune exemption, seulement des seuils. Une exemption ne supprime pas la faille : elle documente la décision de l'accepter, avec un responsable et une échéance.

9. Reporting, alerting et historisation

Rapport	Format	Produit par
---------	--------	-------------

Rapport	Format	Produit par
pytest-report.xml, coverage.xml	JUnit XML, Cobertura XML	Niveau 1 (couverture réutilisée par SonarQube)
gitleaks-report.json, bandit-report.json, trivy-fs.json, checkov-report.json, trivy-image.json, sonarqube-result.json	JSON (artefacts scan-*)	Niveau 2
sbom.cyclonedx.json	CycloneDX	Niveau 2 (Trivy)
summary.json, security-report.md, security-report.html	JSON, Markdown, HTML	Niveau 3
gate-result.json	JSON	Niveau 4
zap-report.html, zap-report.json	HTML, JSON	Niveau 5
Tableau de bord SonarQube	Web (http://localhost:9000)	Niveau 2

Reporting. Le rapport consolidé et la décision du Security Gate (« Security Gate : PASS ») s'affichent directement dans l'onglet *Summary* de chaque exécution GitHub Actions, sans téléchargement.

Alerting. Un message Discord est envoyé en cas de blocage (niveau 1, Security Gate ou ZAP), d'échec du déploiement, et lors d'un déploiement réussi en production, avec le lien vers l'exécution.

Historisation. Chaque exécution conserve ses artefacts dans l'onglet Actions (90 jours par défaut, 3 jours pour l'image Docker). SonarQube garde l'historique des analyses et des statuts d'exemption dans sa base PostgreSQL, ce qui permet de suivre la tendance d'une version à l'autre.

10. Résultats

L'exécution finale sur `main` (run 37863144931, commit 34e8a23, 9 octobre 2026) a réussi ses **14 jobs**, du test unitaire au déploiement en production. Staging et production exécutent la même image et répondent `{"status": "ok"}`.



Figure 9 : exécution finale : niveaux 1 à 6 et alertes au vert, dont le déploiement automatique en production.

Contrôle	Avant (code d'origine)	Après (version finale)	Statut
Pytest	3 tests fonctionnels	6 tests dont 3 de	Vert

Contrôle	Avant (code d'origine)	Après (version finale)	Statut
		sécurité, couverture 89 %	
Gitleaks	Clé Flask, token GitHub et clé AWS en dur	0 secret	Vert
Bandit	7 alertes (3 HIGH, 2 MEDIUM, 2 LOW)	0 (B104 exemptée, EXC-001)	Vert
SonarQube	Gate échoué : sécurité E, 6 vulnérabilités, couverture 0 %	Gate réussi : sécurité A, fiabilité A, couverture 89,2 %	Vert
Trivy fs (SCA)	3 CVE HIGH (Flask, Werkzeug)	0 bloquante	Vert
Checkov (IaC)	3 échecs	0 échec (48 + 320 contrôles)	Vert
Trivy image	python:3.8 en root	0 bloquante (166 signalées, aucune HIGH/CRITICAL corrigeable)	Vert
OWASP ZAP	CSP, X-Frame-Options, X-Content-Type-Options absents	0 FAIL, 63 réussies, 4 avertissements	Vert
Security Gate	—	PASS (et BLOCK sur la version d'origine)	Vert
Déploiement	Aucun	Staging puis production automatiques, durcis, retour arrière testé	Vert

Catégorie OWASP Top 10 (2021)	Faible du projet	Détectée par
A01 Contrôle d'accès défaillant	Protection CSRF absente	SonarQube (S4502)
A02 Défaillances cryptographiques	MD5 pour les mots de passe	Bandit (B324), SonarQube (S4790)
A03 Injection	SQLi, injection de commandes, XSS, SSTI	Bandit (B608, B605), tests Pytest
A05 Mauvaise configuration	Mode debug, en-têtes absents, conteneur root	Bandit (B201), ZAP, Checkov
A06 Composants vulnérables	Flask 2.0.1, Werkzeug 2.0.1, python:3.8	Trivy fs, Trivy image

Catégorie OWASP Top 10 (2021)	Faible du projet	Détectée par
A07 Défaillances d'identification	Clé de session en dur	Gitleaks, Bandit (B105)

Chaque outil couvre une classe de problème différente, et aucun ne remplace les autres : SonarQube Community ne voit pas les injections, Bandit ne voit ni le CSRF ni le Dockerfile, et seul ZAP observe l'application en fonctionnement.

11. Difficultés rencontrées

Symptôme	Cause	Solution
Bandit vert alors que le code contenait une injection SQL	La commande visait <code>app.py</code> à la racine, fichier inexistant : rien n'était analysé	Viser <code>app/</code> ; tester chaque contrôle sur du code vulnérable connu
Pipeline rouge après la correction du code	Les dépendances Flask et Werkzeug étaient toujours obsolètes	Mise à jour et épinglage de toutes les dépendances
4 CVE HIGH dans l'image malgré des dépendances à jour	Bibliothèques embarquées dans pip (<code>urllib3</code> , <code>msgpack</code> , <code>setuptools</code>)	Désinstaller pip à la fin du build
Hooks pre-commit et runner GitHub bloqués par Windows	Smart App Control refuse les exécutables non signés (<code>WinError 4551</code> , <code>Runner.Common.dll</code>)	Hooks lancés via <code>python -m</code> ; Smart App Control désactivé pour le runner
Image déclarée « corrompue » au niveau 5	Téléchargement de l'artefact tronqué (44,0 Mo sur 45,5 Mo) par une connexion instable	Relance du job ; le niveau 6 réutilise l'image déjà chargée au lieu de la retélécharger
Délais dépassés au téléchargement de la base Trivy et des images Docker	Connexion lente	Volume de cache Trivy, relances, contrôle direct du contenu de l'image
Déploiement en production en échec après approbation	Secret de production absent ; une relance réexécute l'ancien commit	Message d'erreur explicite, puis clé générée sur le poste au premier déploiement
Erreurs PowerShell sur des commandes Docker réussies	Windows PowerShell 5.1 transforme la sortie d'erreur en exception avec <code>ErrorActionPreference = stop</code>	<code>ErrorActionPreference = Continue</code> et contrôle de <code>\$LASTEXITCODE</code>
Le runner auto-hébergé expose le poste	Dépôt public : une pull request externe pourrait exécuter du code sur le poste	Jobs locaux limités aux push et PR internes, approbation des contributeurs externes

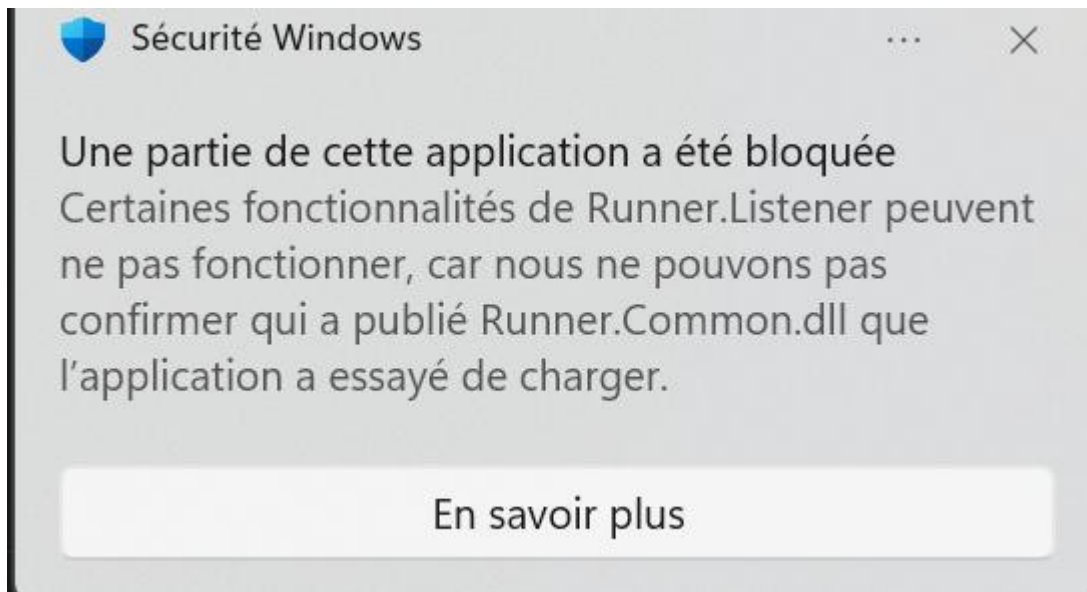


Figure 10 : Windows Smart App Control bloque Runner.Common.dll lors de la configuration du runner auto-hébergé.

12. Axes d'amélioration

- Corriger le risque accepté EXC-002 en ajoutant Flask-WTF (CSRFProtect) et un jeton CSRF sur toutes les requêtes POST.
- Transporter l'image par GitHub Container Registry et la déployer par son empreinte (*digest*) : `docker pull` vérifie chaque couche et reprend un téléchargement interrompu.
- Signer l'image (Cosign) et attacher le SBOM comme attestation.
- Fixer les versions des outils et des images (`trivy:latest`, `zapproxy:stable...`), voire par empreinte, pour des résultats reproductibles.
- Publier les résultats au format SARIF dans l'onglet *Security* de GitHub.
- Remplacer le scan ZAP baseline par un scan actif avec parcours authentifié (connexion puis `/notes`).
- Ajouter la surveillance de l'exécution : Falco pour les conteneurs, Grafana ou Wazuh pour les journaux et les alertes.
- Protéger la branche `main` : pull request obligatoire et jobs du pipeline requis avant fusion.
- Remplacer le runner sur poste personnel par une machine dédiée, toujours allumée.

13. Conclusion

Le projet livre une chaîne DevSecOps complète et fonctionnelle : contrôles sur le poste, tests, détection de secrets, SAST avec deux outils complémentaires, SCA, scan IaC, scan d'image et SBOM, rapport consolidé, Security Gate central, staging attaqué par OWASP ZAP, déploiement automatique en production avec retour arrière, et alertes Discord. La dernière exécution a réussi ses 14 jobs, et l'application déployée est l'image exacte qui a été analysée.

Au-delà de la détection, nous avons corrigé les failles : l'application de départ comptait 10 failles volontaires, et la version finale passe tous les contrôles avec deux exemptions documentées et datées. Le test négatif confirme que le Security Gate bloque toujours la version vulnérable d'origine.

Le projet a surtout montré qu'un pipeline de sécurité repose sur des décisions : quoi bloquer, à quel seuil, comment traiter un faux positif ou un risque accepté, et comment vérifier qu'un contrôle

fonctionne vraiment. Un contrôle vert qui n'analysait rien, comme Bandit au début du projet, donne un faux sentiment de sécurité ; c'est pourquoi chaque contrôle doit être testé sur du code vulnérable connu.

Annexe A. Commandes de reproduction

Commandes pour Windows PowerShell avec Docker Desktop, à lancer dans le dossier du dépôt.

```
# Contrôles avant commit (une seule fois)
pip install pre-commit
pre-commit install
pre-commit run --all-files

# Tous les scanners + rapport + Security Gate sur le poste
.\scripts\local-scan.ps1

# Serveur SonarQube (tableau de bord : http://localhost:9000)
docker compose -f sonarqube/docker-compose.yml up -d

# Secrets
docker run --rm -v "${PWD}:/repo" zricethezav/gitleaks:latest dir /repo --redact

# SCA (dépendances)
docker run --rm -v "${PWD}:/work" -w /work aquasec/trivy:latest fs --scanners vuln
--severity HIGH,CRITICAL --ignore-unfixed .

# IaC (Dockerfile + workflows)
docker run --rm -v "${PWD}:/src" -w /src bridgecrew/checkov:latest --config-file
.checkov.yaml

# Runner auto-hébergé (laisser la fenêtre ouverte)
cd C:\actions-runner
.\run.cmd

# Livrer une modification et déclencher le pipeline
git add -A
git commit -m "message"
git push

# Vérifier les environnements
Invoke-WebRequest http://127.0.0.1:5001/health -UseBasicParsing # staging
Invoke-WebRequest http://127.0.0.1:8080/health -UseBasicParsing # production
```

Annexe B. Fichiers de configuration

Les fichiers complets sont dans le dépôt ; voici les principaux, tels qu'ils sont versionnés.

B.1 Orchestrateur `.github/workflows/devsecops.yml`

```
name: DevSecOps Pipeline

on:
  push:
    branches: [main]
  pull_request:
  workflow_dispatch:

permissions:
  contents: read
```

```

concurrency:
  group: devsecops- $\{\{\text{github.ref}\}\}$ 
  cancel-in-progress: false

jobs:
  build-test:
    name: "1 Build & Test"
    uses: ../github/workflows/level1-build-test.yml

  security-scans:
    name: "2 Security Scans"
    needs: build-test
    uses: ../github/workflows/level2-security-scans.yml
    secrets: inherit

  reports:
    name: "3 Report Generation"
    needs: [build-test, security-scans]
    # S'exécute même si un scanner a planté : le gate verra un rapport manquant et
    bloquera.
    if:  $\{\{\text{!cancelled() \&\& needs.build-test.result == 'success'}\}\}$ 
    uses: ../github/workflows/level3-reports.yml

  security-gate:
    name: "4 Security Gate"
    needs: reports
    if:  $\{\{\text{!cancelled() \&\& needs.reports.result == 'success'}\}\}$ 
    uses: ../github/workflows/level4-security-gate.yml

  staging-dast:
    name: "5 Staging + DAST"
    needs: security-gate
    # Déploiement uniquement depuis main, jamais depuis une pull request.
    if:  $\{\{\text{github.ref == 'refs/heads/main' \&\& github.event_name != 'pull_request'}\}\}$ 
  }
  uses: ../github/workflows/level5-staging-dast.yml

  deploy-production:
    name: "6 Deploy to Production"
    needs: staging-dast
    uses: ../github/workflows/level6-deploy-production.yml

  notify:
    name: "Alerts"
    needs: [build-test, security-gate, staging-dast, deploy-production]
    if:  $\{\{\text{always()}\}\}$ 
    uses: ../github/workflows/notify-discord.yml
    with:
      build:  $\{\{\text{needs.build-test.result}\}\}$ 
      gate:  $\{\{\text{needs.security-gate.result}\}\}$ 
      staging:  $\{\{\text{needs.staging-dast.result}\}\}$ 
      deploy:  $\{\{\text{needs.deploy-production.result}\}\}$ 
    secrets: inherit

```

B.2 Politique du Security Gate `security-policy.json`

```

{
  "fail_if_report_missing": true,
  "tools": {

```

```

    "gitleaks": { "block_severities": ["CRITICAL"] },
    "bandit": { "block_severities": ["MEDIUM", "HIGH"] },
    "sonarqube": { "block_severities": ["CRITICAL"] },
    "trivy-fs": { "block_severities": ["HIGH", "CRITICAL"],
"block_cvss_at_or_above": 7.0, "only_fixable": true },
    "checkov": { "block_severities": ["LOW", "MEDIUM", "HIGH", "CRITICAL"] },
    "trivy-image": { "block_severities": ["HIGH", "CRITICAL"],
"block_cvss_at_or_above": 7.0, "only_fixable": true }
  }
}

```

Chaque secret Gitleaks et chaque échec du Quality Gate SonarQube est converti en constat CRITICAL par le rapport consolidé ; les libellés de chaque outil sont omis ici.

B.3 Exemption Bandit `bandit.yaml`

```

# EXC-001 | B104 hardcoded_bind_all_interfaces | app/app.py
# Reason : false positive. The app runs inside a container, where binding to
#         0.0.0.0 is required for the port mapping to work.
# Approved : 2026-10-08, review by 2027-04-08
skips:
  - B104

exclude_dirs:
  - tests
  - venv

```

B.4 Règles OWASP ZAP `.zap/rules.tsv`

```

10020 FAIL (Missing Anti-clickjacking Header)
10021 FAIL (X-Content-Type-Options Header Missing)
10038 FAIL (Content Security Policy Header Not Set)

```